

COMP90050 ADBS

Week 6

Q1 ACID Isolation

1. Discuss why the isolation property of ACID properties will apply to both an Online shopping platform as well as an Online banking system despite that they are different applications dealing with different data.

Isolation- transaction are executed as if it is the only one in the system.

- For example, in an application that transfers funds from one account to another, the isolation ensures that another transaction sees the transferred funds in one account or the other, but not in both, nor in neither.
- Both systems may need to serve a **large number of customers (& transactions)** at any given time
- To achieve **consistency**, a transaction should not use the values that have been modified by another uncommitted transaction
 1. Serial schedule ensures consistency, but handling one customer at a time is *inefficient*
 2. System should allow concurrent transactions, but changes to the data should be made as if the transactions run on a serial schedule
- **Objective of Isolation:** transaction runs as if it is the only transaction in the system and does not become aware of other concurrent transactions

Isolation helps with achieving a high level of efficiency while maintaining the consistency of the data that is manipulated

Q2 Nested Transaction Rules

2. In a nested transaction, a transaction PARENT has three sub-transactions A, B, and C. For each of the following scenarios, answer which of these four transactions' commits can be made durable, and which ones have to be forced to rollback.

- Scenario 1: Commit by A, B, and C; but PARENT rolls back.
- Scenario 2: Commit by A, B, C, and PARENT.
- Scenario 3: Commit by A, B, and PARENT; but C rolls back.

Commit rule

- A subtransaction can either commit or abort, however, **commit cannot take place unless the parent itself commits.**
- Subtransactions have A, C, and I properties but not D property unless all its ancestors commit.
- Commit of a sub transaction makes its results available only to its parents.

Roll back Rules

If a subtransaction rolls back, all its children are forced to roll back.

- **Scenario 1:** All four transactions roll back, with no durable commit by any transaction
- **Scenario 2:** All four transactions make durable commits, with no roll back
- **Scenario 3:** Durable commits by A, B, and Parent; roll back by C only

Q3 TP Monitor Services

3. Discuss why using a single process for all the transaction processing monitor services is not a good idea.

One error in the process can impact all the transactions.

Leading to poor performance, and unable to distribute the monitoring of different transactions

Q4 Flat Transactions

4. We have seen flat transactions in class. The following flat transaction definition creates a problem for the system and shows why flat transactions alone cannot be used. Please discuss the example and explain the associated issue:

```
GiveEndOfYearBonus()  
{  real bonus;  
  receive(bonus);  
  exec sql BEGIN WORK;  
    exec SQL UPDATE customer  
      set account = account + :bonus  
  exec sql COMMIT WORK;  
}
```

The customer table could be a **large table** with millions of customers.

The transaction can potentially be **doing a lot of work** during its execution and **take a long time**.

Any failure in this transaction's execution would mean **a lot of work lost**.

Q5 Singleton Select & Cursor

5. What is the difference between a singleton select and a cursor?

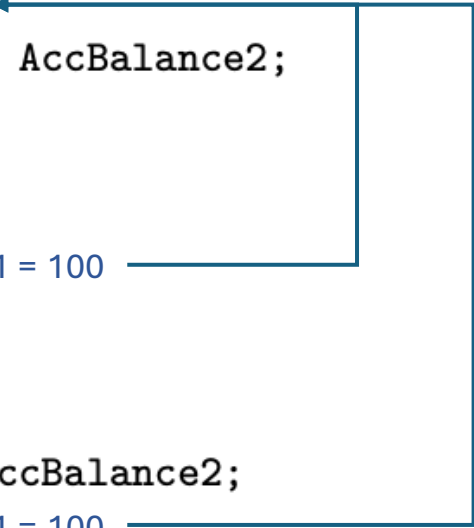
Singleton select: a SQL statement that returns only a single row of data

Cursor: If more than one rows are returned, then a 'cursor' is returned, pointing at the first row of the results. The cursor can be used to traverse all the results rows one by one.

Q6 Save Points

6. What is the value of **AccBalance1** after running the following transaction operations? Discuss why save points are needed.

```
BEGIN WORK
SAVE WORK 1
AccBalance1 = 100;
AccBalance2 = 50;
SAVE WORK 2 AccBalance1 = 100
AccBalance3 = AccBalance1 + AccBalance2;
AccBalance1 = 0;
AccBalance2 = 0;
SAVE WORK 3 AccBalance1 = 0
ROLLBACK WORK (2) AccBalance1 = 100
AccBalance3 = AccBalance1;
AccBalance1 = 0;
SAVE WORK 4 AccBalance1 = 0
AccBalance3=AccBalance3 + AccBalance2;
ROLLBACK WORK (2) AccBalance1 = 100
AccBalance2 = AccBalance2 + 100;
COMMIT WORK AccBalance1 = 100
```



Without save points, the transaction needs to be started from scratch every time it needs to rollback.

Q7 Quadtree

7. In the following quadtree the division of space is based on a rule that each quadrant could only have at most one data point. Based on that we can see that the quadtree is not properly formed. Please finish the space subdivision process for this tree.

